Universidad de Costa Rica

Escuela de Ciencias de la Computación e Informática

CI-0141 | G01

BASES DE DATOS AVANZADAS

Laboratorio 8

Bases de Datos NoSQL - Orientadas a Familias de Columnas

Profesor:

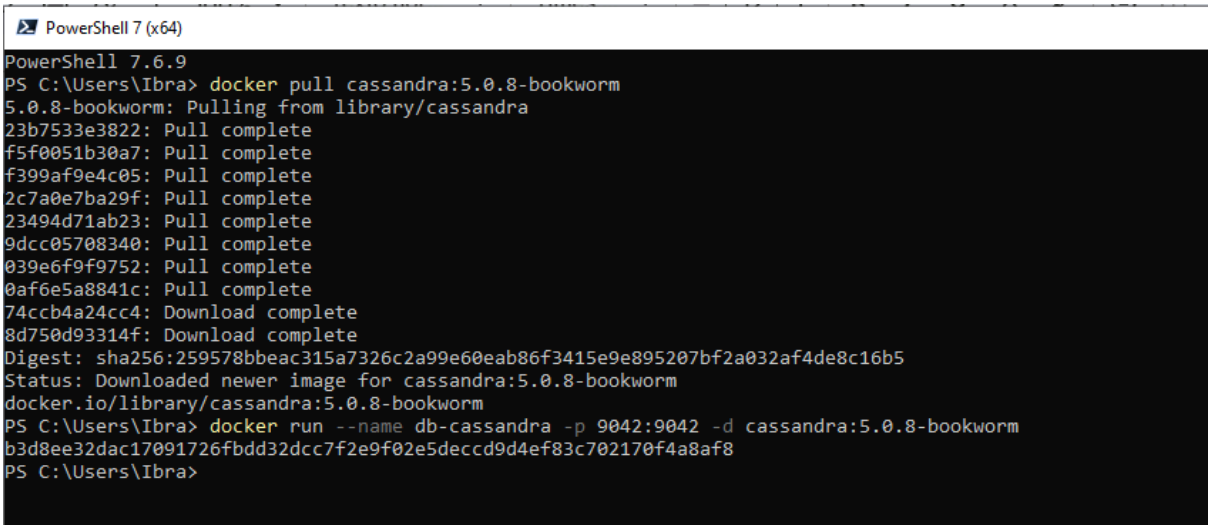
Mag. Luis Carlos Salas

Javier Cruz Ruiz C02517

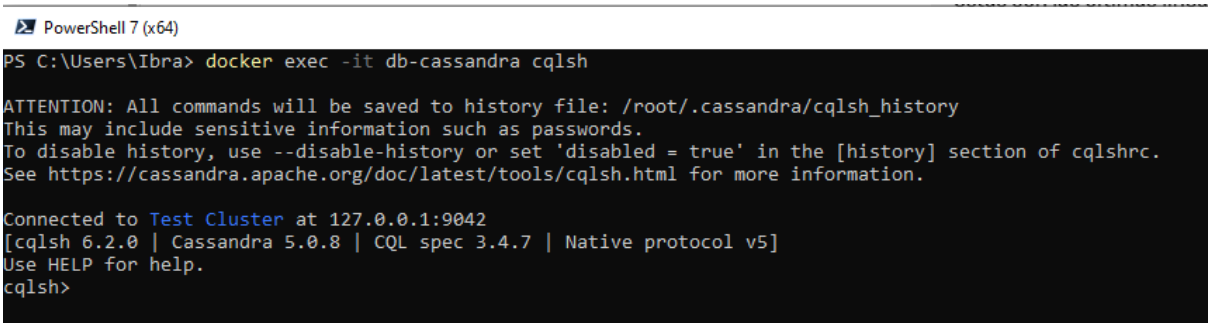
19 de Noviembre, 2026

Creación de instancia

1. Realice los siguientes pasos para realizar la habilitación de una instancia de Cassandra utilizando un contenedor en Docker.

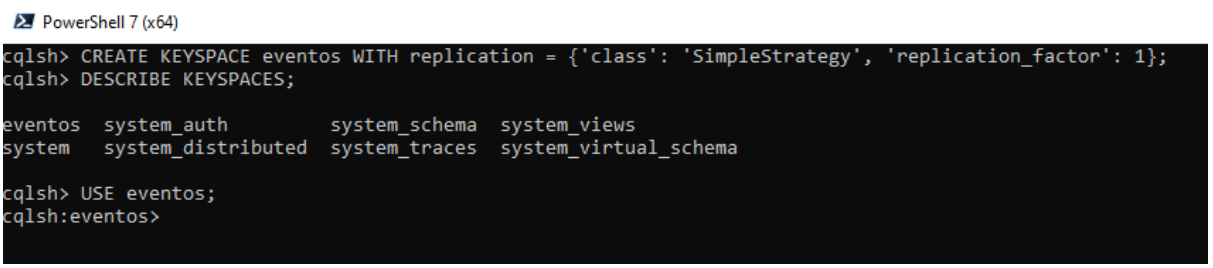


2. Establezca una conexión la instancia de Cassandra utilizando la línea de comando.



Creación de keyspace

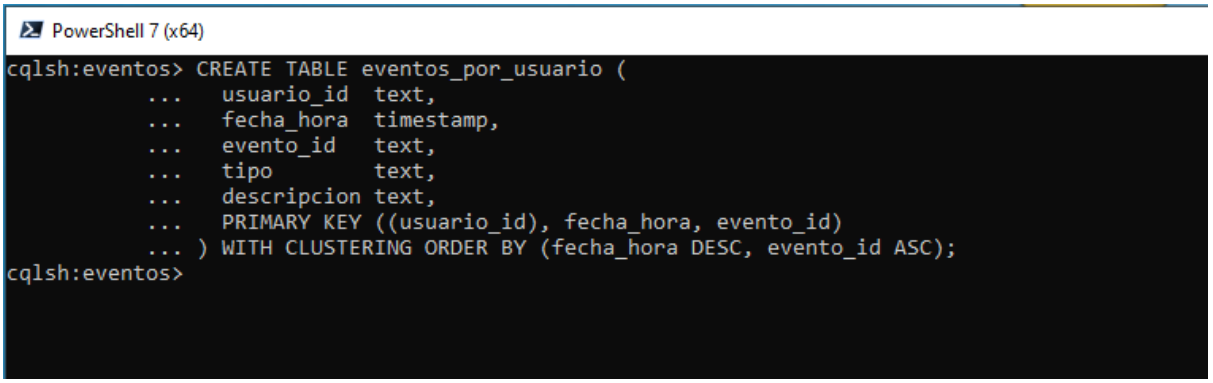
1. Cree un keyspace llamado “eventos” utilizando una estrategia de replicación adecuada para un ambiente local con un solo nodo. Incluya el código CQL utilizado.



Creación de tables

1. A partir de estas consultas, diseñe las tablas necesarias para almacenar y consultar los eventos de actividad de usuarios.

a. Proponga y cree una tabla para consultar eventos por usuario. La tabla debe permitir recuperar los eventos asociados a un usuario específico y consultar los eventos de ese usuario dentro de un rango de fechas. Además, se debe considerar la fecha y hora del evento para organizar los resultados del más reciente al más antiguo. Por ejemplo, consultar todos los eventos del usuario USR-001, ordenados del más reciente al más antiguo. Para esta tabla, indique: nombre y columnas, primary key, considerando la partition key y las clustering keys. Finalmente, justifique su diseño e incluya el código CQL utilizado

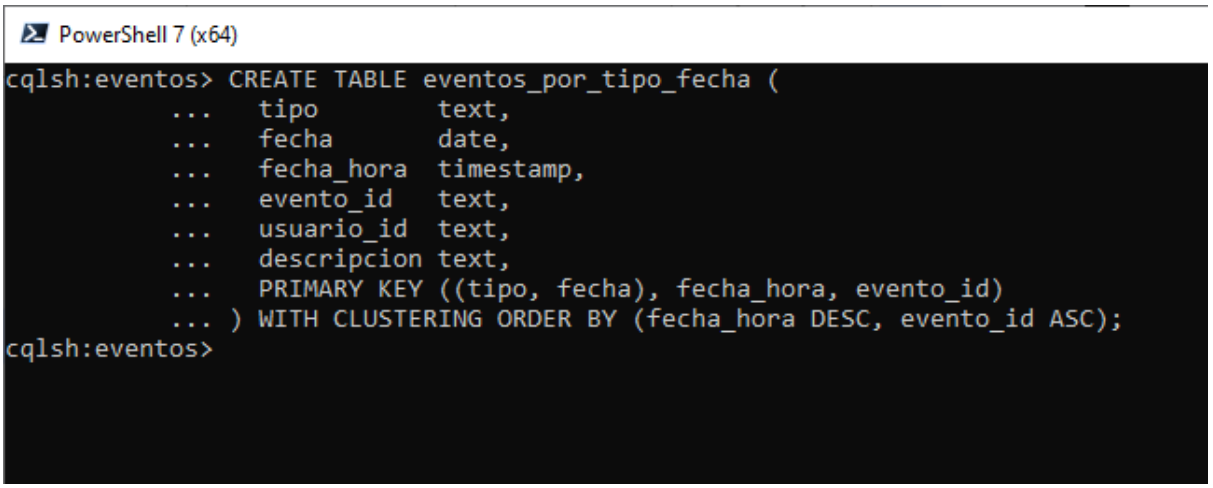


- Nombre: eventos_por_usuario
- Columnas: usuario_id, fecha_hora, evento_id, tipo, descripcion
- Primary Key: usuario_id, fecha_hora, evento_id
- Partition key: usuario_id
- Clustering keys: fecha_hora (DESC), evento_id (ASC)

Justificación: Se utiliza usuario_id como partition key porque todas las consultas de esta tabla se filtrarán por usuario. fecha_hora se utiliza como clustering key con orden descendiente.

Cassandra guarda las filas ya ordenadas dentro de la partición. evento_id también es clustering key para garantizar unicidad y evitar que dos eventos se sobrescriban entre sí.

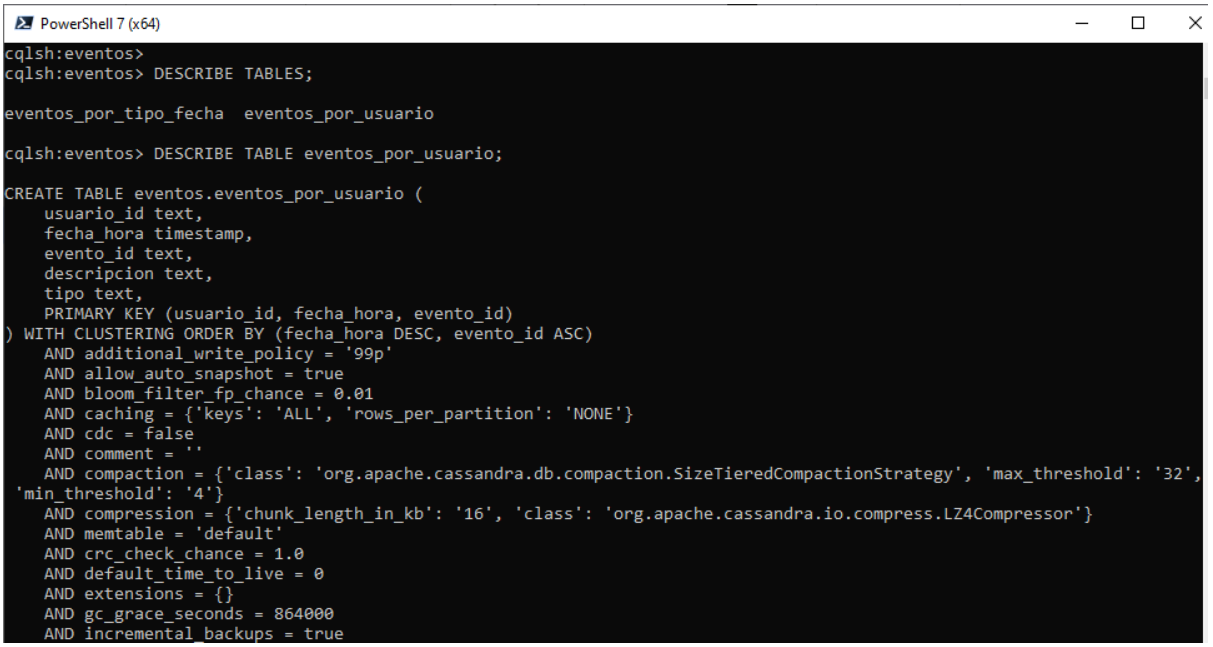
b. Proponga y cree una tabla para consultar eventos por tipo de evento y fecha. La tabla debe permitir recuperar los eventos de un tipo específico ocurridos en una fecha determinada. Por ejemplo, consultar todos los eventos de tipo LOGIN registrados el día 2000-05-15. Para efectos de este ejercicio, la columna fecha representa la fecha calendario del evento, sin considerar la hora. La fecha y hora exacta en que ocurrió el evento debe conservarse en una columna diferente. Para esta tabla, indique: nombre y columnas, primary key, considerando la partition key y las clustering keys. Finalmente, justifique su diseño e incluya el código CQL utilizado.

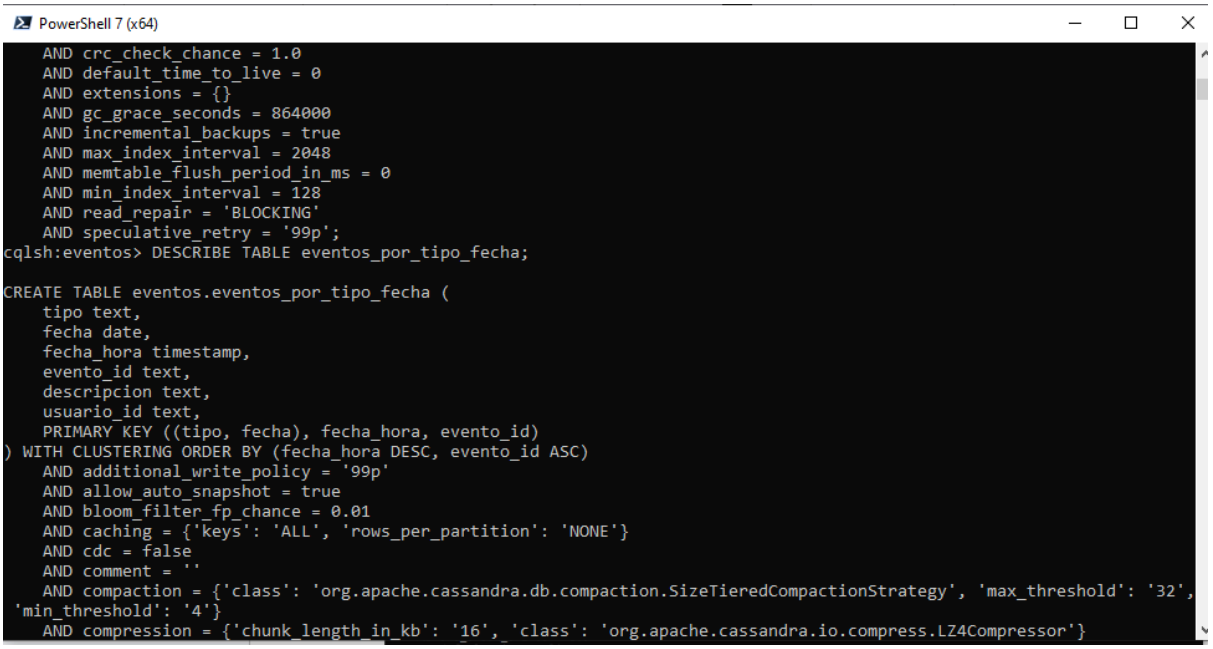


- Nombre: eventos_por_tipo_fecha
- Columnas: tipo, fecha, fecha_hora, evento_id, usuario_id, descripcion
- Primary Key: tipo, fecha, fecha_hora, evento_id
- Partition key: (tipo, fecha)
- Clustering keys: fecha_hora (DESC), evento_id (ASC)

Justificación: (tipo, fecha) funciona como partition key para la consulta “eventos LOGIN del 2000-05-15”. Al incluir la fecha en el partition key se evita una partición más grande, con (tipo, fecha) hay una partición por tipo y por día, de tamaño acotado. fecha es de tipo date

(solo día) y fecha_hora conserva el momento exacto, fecha_hora y evento_id son clustering keys para ordenar los eventos del día y evitar que se sobrescriban.





2. Conteste las siguientes preguntas:

a. ¿Se pueden resolver las consultas por usuario con una misma tabla? Justifique su respuesta.

Sí. Las dos consultas utilizan usuario_id como filtro principal. Al ser la partition key, permite obtener todos los eventos de un usuario o filtrarlos por un rango de fechas mediante

fecha_hora, que funciona como clustering key. Por eso, solo cambia la condición del WHERE, no la estructura de la tabla.

b. ¿Por qué la consulta por tipo de evento y día requiere una tabla diferente?

En Cassandra, los datos se organizan y consultan principalmente mediante la partition key. En la tabla por usuario, los eventos están agrupados por usuario_id, por lo que buscar todos los LOGIN de un día implicaría recorrer todas las particiones, algo que Cassandra no permite por defecto. Para hacer esa consulta de forma eficiente, se necesita otra tabla organizada por (tipo, fecha).

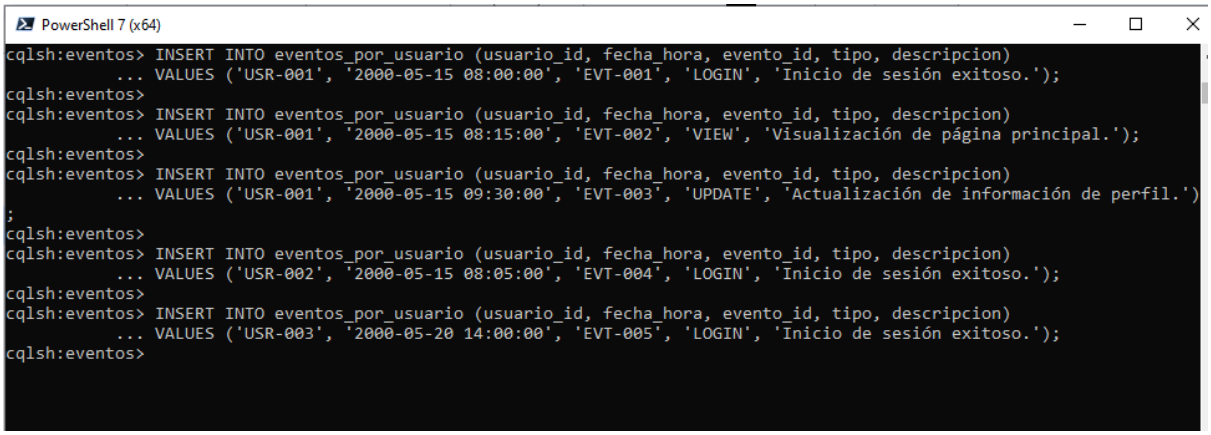
c. ¿Es necesario almacenar los eventos en ambas tablas? Justifique su respuesta.

Sí. En Cassandra no hay joins y cada tabla está diseñada para un tipo de consulta. Por eso, cada evento debe guardarse en ambas tablas. Esto sería desnormalización, se duplican los datos para obtener consultas más rápidas. El costo es que la aplicación debe mantener ambas tablas actualizadas. Para hacerlo, al insertar un evento se ejecutan dos INSERT, que también pueden agruparse en un BATCH.

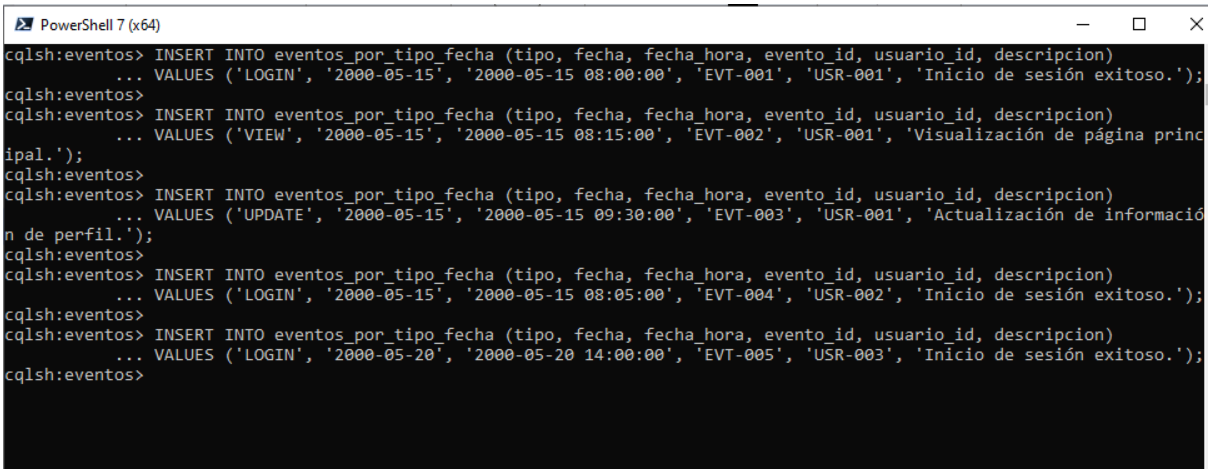
Inserción de datos

1. Inserte los siguientes eventos de actividad de usuarios.

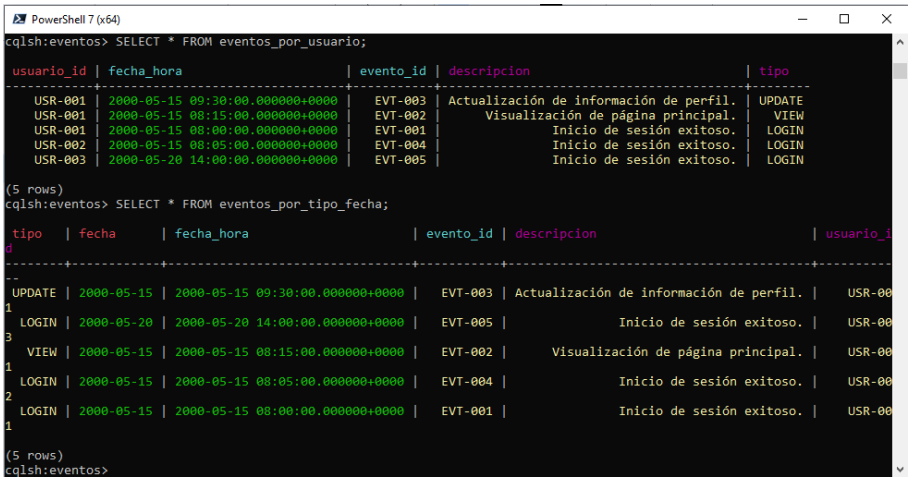
Inserción en eventos_por_usuario:



Inserción en eventos_por_tipo_fecha:



2. Consulte ambas tablas para verificar que los datos fueron insertados correctamente.



3. Incluya en el reporte el código CQL utilizado y tome una captura de pantalla que evidencien la inserción de los datos.

```
INSERT INTO eventos_por_usuario (usuario_id, fecha_hora,
evento_id, tipo, descripcion)
VALUES ('USR-001', '2000-05-15 08:00:00', 'EVT-001', 'LOGIN',
'Inicio de sesión exitoso.');

INSERT INTO eventos_por_usuario (usuario_id, fecha_hora,
evento_id, tipo, descripcion)
VALUES ('USR-001', '2000-05-15 08:15:00', 'EVT-002', 'VIEW',
'Visualización de página principal.');
```

```
INSERT INTO eventos_por_usuario (usuario_id, fecha_hora,
evento_id, tipo, descripcion)
VALUES ('USR-001', '2000-05-15 09:30:00', 'EVT-003', 'UPDATE',
'Actualización de información de perfil.');

INSERT INTO eventos_por_usuario (usuario_id, fecha_hora,
evento_id, tipo, descripcion)
VALUES ('USR-002', '2000-05-15 08:05:00', 'EVT-004', 'LOGIN',
'Inicio de sesión exitoso.');

INSERT INTO eventos_por_usuario (usuario_id, fecha_hora,
evento_id, tipo, descripcion)
VALUES ('USR-003', '2000-05-20 14:00:00', 'EVT-005', 'LOGIN',
'Inicio de sesión exitoso.');
```

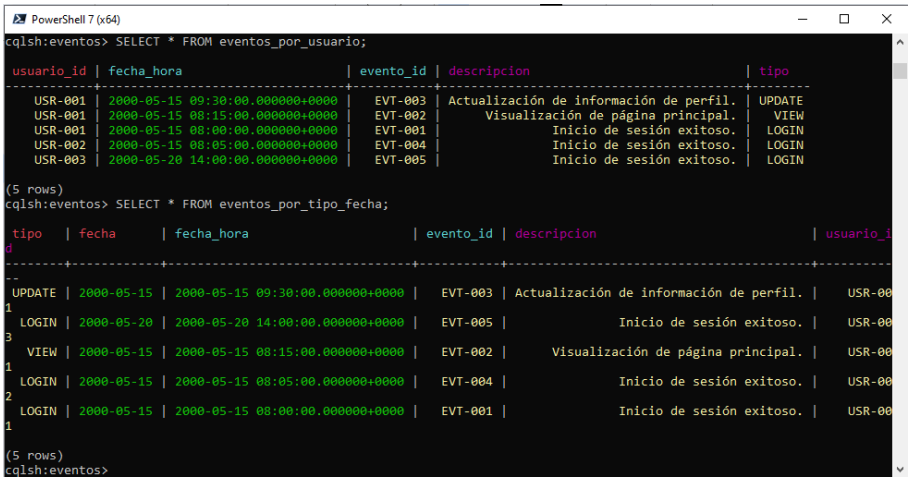
```
INSERT INTO eventos_por_tipo_fecha (tipo, fecha, fecha_hora,
evento_id, usuario_id, descripcion)
VALUES ('LOGIN', '2000-05-15', '2000-05-15 08:00:00', 'EVT-001',
'USR-001', 'Inicio de sesión exitoso.');

INSERT INTO eventos_por_tipo_fecha (tipo, fecha, fecha_hora,
evento_id, usuario_id, descripcion)
VALUES ('VIEW', '2000-05-15', '2000-05-15 08:15:00', 'EVT-002',
'USR-001', 'Visualización de página principal.');

INSERT INTO eventos_por_tipo_fecha (tipo, fecha, fecha_hora,
evento_id, usuario_id, descripcion)
VALUES ('UPDATE', '2000-05-15', '2000-05-15 09:30:00', 'EVT-003',
'USR-001', 'Actualización de información de perfil.');

INSERT INTO eventos_por_tipo_fecha (tipo, fecha, fecha_hora,
evento_id, usuario_id, descripcion)
VALUES ('LOGIN', '2000-05-15', '2000-05-15 08:05:00', 'EVT-004',
'USR-002', 'Inicio de sesión exitoso.');

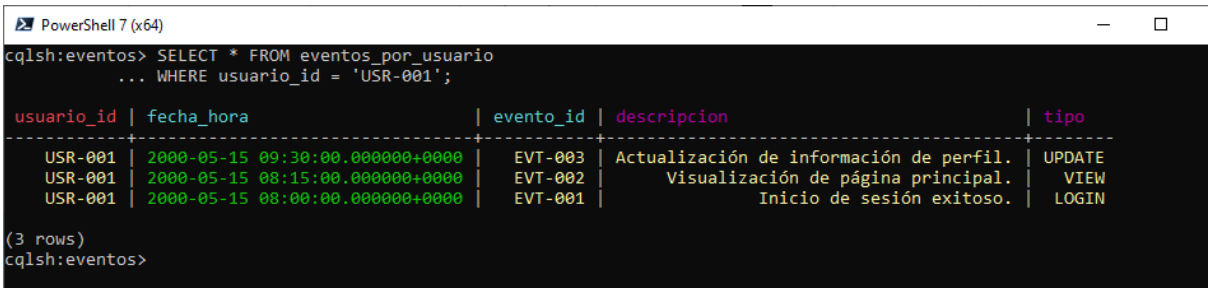
INSERT INTO eventos_por_tipo_fecha (tipo, fecha, fecha_hora,
evento_id, usuario_id, descripcion)
VALUES ('LOGIN', '2000-05-20', '2000-05-20 14:00:00', 'EVT-005',
'USR-003', 'Inicio de sesión exitoso.');
```



Consulta de datos

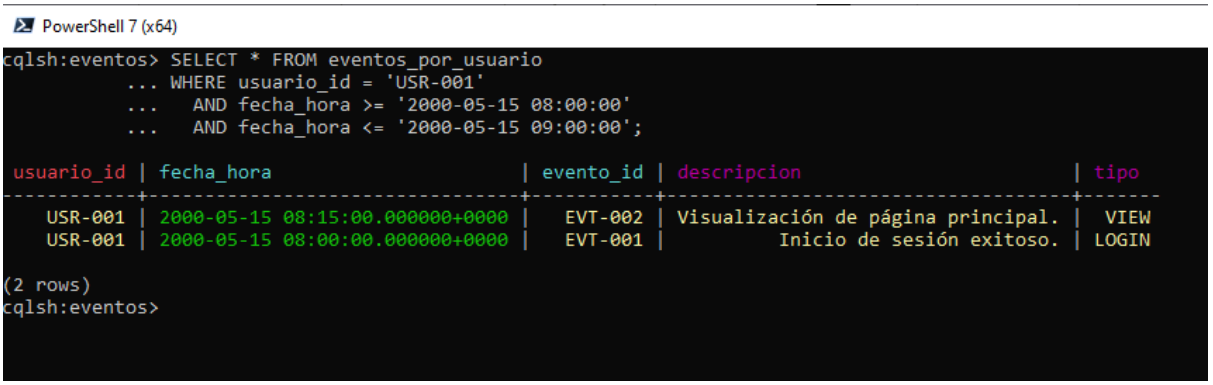
1. Consulte todos los eventos de un usuario específico. La consulta debe recuperar todos los eventos asociados al usuario USR-001. El resultado debe mostrar los eventos ordenados del más reciente al más antiguo.

```
SELECT * FROM eventos_por_usuario
WHERE usuario_id = 'USR-001';
```



2. Consulte los eventos de un usuario específico dentro de un rango de fechas. La consulta debe recuperar los eventos del usuario USR-001 ocurridos entre 2000-05-15 08:00:00 y 2000-05-15 09:00:00.

```
SELECT * FROM eventos_por_usuario
WHERE usuario_id = 'USR-001'
  AND fecha_hora >= '2000-05-15 08:00:00'
  AND fecha_hora <= '2000-05-15 09:00:00';
```



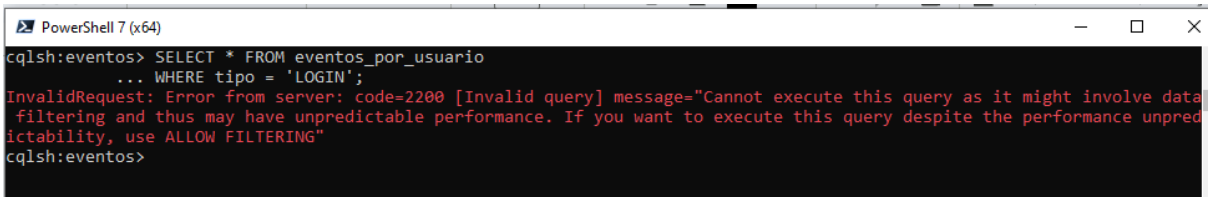
3. Consulte los eventos de tipo LOGIN ocurridos en una fecha específica. La consulta debe recuperar todos los eventos de tipo LOGIN registrados en la fecha 2000-05-15.

```
SELECT * FROM eventos_por_tipo_fecha
WHERE tipo = 'LOGIN'
  AND fecha = '2000-05-15';
```



4. Intente realizar una consulta que filtre por una columna que no forme parte de la primary key de la tabla utilizada. Por ejemplo, intente consultar la tabla orientada a eventos por usuario filtrando únicamente por el tipo de evento LOGIN, sin utilizar la partition key de esa tabla. Tome una captura de pantalla del error generado y comente por qué Cassandra no permite ejecutar directamente esa consulta.

```
SELECT * FROM eventos_por_usuario
WHERE tipo = 'LOGIN';
```



Cassandra no permite esta consulta directamente porque tipo no forma parte de la primary key de eventos_por_usuario. Como los datos se buscan mediante usuario_id, al no indicar esa clave tendría que revisar todas las particiones del cluster. Esto resulta costoso y afecta el rendimiento a medida que crecen los datos. Por eso, Cassandra exige ALLOW FILTERING para permitir la consulta, aunque no se recomienda usarlo en producción.

Eliminación de instancia

1. Ejecute los siguientes comandos para eliminar el contenedor de la instancia de Cassandra. Asegúrese de haber terminado esta parte del laboratorio, ya que la eliminación no se puede deshacer.

